

**PROJECT: EASYSHARE**  
**DESIGN DOCUMENT**



# Design Document

for the

# EasyShare System

GLOBAL RTS

## PROJECT: EASYSHARE DESIGN DOCUMENT



### Content

|                                           |   |
|-------------------------------------------|---|
| 1. System Overview .....                  | 3 |
| 2. System Architecture .....              | 3 |
| 3. Functional Module Designs .....        | 3 |
| 3.1. Login and Authentication .....       | 3 |
| 3.2. Home Page Functions .....            | 4 |
| 3.3. Help Page .....                      | 5 |
| 4. Database Design (Simplified ERD) ..... | 5 |
| 5. User Interface Design .....            | 6 |
| 5.1. Navigation Layout: .....             | 6 |
| 5.2. Modal Dialogs: .....                 | 7 |
| 6. Permissions Model .....                | 7 |
| 7. Design Considerations .....            | 7 |

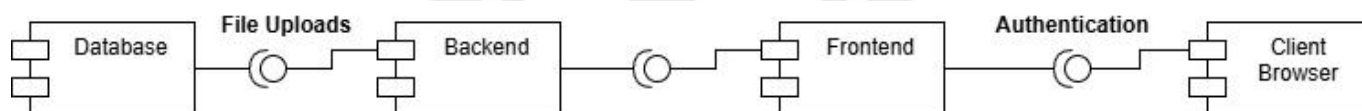
## PROJECT: EASYSHARE DESIGN DOCUMENT



# 1. System Overview

EasyShare is a web-based file management system that allows users to efficiently manage digital content through folders and files. It supports user authentication, file organization, access control, and administrative file recovery in a user-friendly interface.

# 2. System Architecture



*Diagram 1: High-Level Component Diagram*

- **Frontend:** HTML, CSS (Bootstrap 5), JavaScript (vanilla or framework-based)
- **Backend:** Node.js with Express.js
- **Database:** PostgreSQL or MongoDB (for flexibility in file structure)
- **Authentication:** Session-based or JWT
- **File Uploads:** Multer for handling file uploads

# 3. Functional Module Designs

## 3.1. Login and Authentication

- Login Page: Username and password input
- Backend Logic:
  - ❖ Validate credentials against the database
  - ❖ Establish session or return JWT token
  - ❖ Role detection (e.g., admin vs standard user)

## PROJECT: EASYSHARE DESIGN DOCUMENT



- Security:
  - ❖ Passwords hashed using bcryptjs
  - ❖ CSRF and XSS protections enabled

### 3.2. Home Page Functions

- Folder Management
  - ❖ Create Folder: User inputs a name; backend stores metadata and creates logical path
  - ❖ Remove Folder: Marks folder as deleted (soft delete)
  - ❖ Search Folder: Filters folders by name using search input
  - ❖ Set Visibility:
    - public, private (role-based access control)
    - Stored as part of folder metadata
- File and Subfolder Management
  - ❖ Upload File / Add Subfolder: Files uploaded using Multer and linked to a parent folder ID
  - ❖ Move File / Subfolder:
    - Drag-and-drop UI or dropdown selector
    - Backend updates the parent folder reference
  - ❖ Remove File: Soft delete with timestamp
  - ❖ Search Files/Subfolders:

## PROJECT: EASYSHARE DESIGN DOCUMENT



- Real-time search using input field
- Case-insensitive partial matching
- ❖ Set Visibility Rights:
  - Each file or subfolder can have its own access control
  - Admins can override access restrictions
- Deleted Files Management (Admin Only)
  - ❖ Recycle Cash View:
    - Displays deleted files and folders
    - Only accessible by system administrators
  - ❖ Restore File:
    - Reverts the deleted flag and returns file to location chose by user

### 3.3. Help Page

- Static or dynamically loaded help content
- Sections:
  - ❖ Main Page
  - ❖ Helper

## 4. Database Design (Simplified ERD)

## PROJECT: EASYSHARE DESIGN DOCUMENT

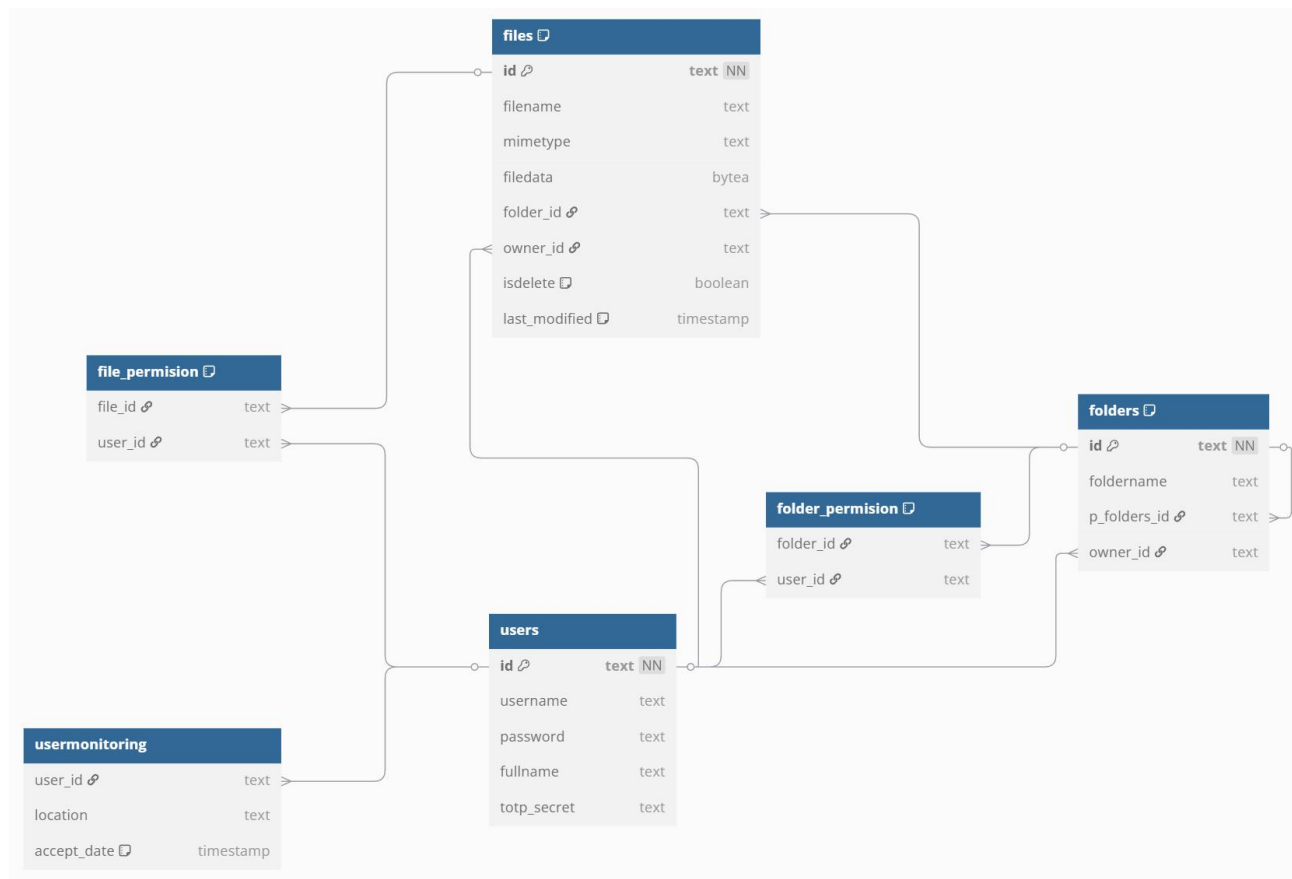


Diagram 2: Entity-Relationship Diagram

## 5. User Interface Design

### 5.1. Navigation Layout:

- Top navbar: Main page | Helper | Log out
- Main page:
  - ❖ List of all root folders
  - ❖ Create/Delete Folder button
  - ❖ Cash button
  - ❖ Search and Filter tools

## PROJECT: EASYSHARE DESIGN DOCUMENT



### 5.2. Modal Dialogs:

- Current folder content
- Create/Move/Delete Sub-Folders
- Upload/Move/Delete Files
- Set Visibility
- Confirm Delete or Restore

## 6. Permissions Model

| Action                | User | Admin |
|-----------------------|------|-------|
| View folders/files    | ✓    | ✓     |
| Add folders/files     | ✓    | ✓     |
| Delete folders/files  | ✓    | ✓     |
| Move folders/files    | ✓    | ✓     |
| Restore deleted files | ✗    | ✓     |
| Set visibility rights | ✓    | ✓     |
| Access Help           | ✓    | ✓     |

*Table 1: Permissions model*

## 7. Design Considerations

- **Scalability:** Folder hierarchy can be nested deeply; recursive querying supported.
- **Performance:** Indexed searches; lazy loading for folder contents.
- **Accessibility:** ARIA labels and keyboard support for major actions.
- **Responsiveness:** Layout adapts to mobile and tablet screen sizes.